

What is UNIPLEX (MIST EMS)?

UNIPLEX at MIST is an integrated **Education Management System** used for tasks like:

- Course registration
- Attendance tracking
- Result publishing
- Assignment submission
- Faculty-student communication
- Class routine management

How Task Analysis Applies to UNIPLEX ???

1. Task Decomposition (HTA) in UNIPLEX
2. Knowledge-Based Task Analysis
3. Entity/Object-Based Analysis

Task Decomposition (HTA) in UNIPLEX

Task: Register for courses

Hierarchical Task Analysis (HTA):

Main Goal: Register for the semester

Subtasks:

1. Log in to UNIPLEX
2. Navigate to course registration panel
3. Select courses as per credit requirement
4. Confirm and submit
5. Download or verify course list

Plan: Do 1 → 2 → 3 → 4 → 5 in that order.

This breakdown helps system designers ensure the interface supports the task sequence logically and clearly.

Knowledge-Based Task Analysis

Task: Register for courses

To successfully register or use UNIPLEX, a user (student) needs to **know**:

- Their credit requirements
- Course prerequisites
- How to navigate the UNIPLEX dashboard
- Course codes and titles
- Deadline for registration

Designing the system should consider this knowledge – e.g., providing tooltips, course descriptions, credit summaries.

Entity/Object-Based Analysis

Task: Register for courses

In UNIPLEX, key **entities (objects)** and **actions** include:

This helps map the system's data models and understand how users interact with features.

Entity/Object

Student profile

Courses

Assignments

Results

Attendance

Associated Actions

View, update, submit

Browse, select, drop

Upload, view feedback

View, download, contest

Check, notify instructor

Classroom Example for Discussion

Example Task: Submitting an Assignment via UNIPLEX

Asking:

- What steps do you follow?
- What knowledge is required?
- What objects and actions are involved?

Expected Answers:

- Steps:** Login → Go to course → Find assignment → Upload → Confirm
- Knowledge:** Deadline, file format, assignment title
- Objects/Actions:** Assignment (upload), Course (navigate), Notification (receive)

Then, let students **draw an HTA tree** or flow diagram of this process.

Summary Table

Task Analysis Method

Task Decomposition

Knowledge-Based

Entity/Object-Based

UNIPLEX Example

Registering courses, uploading assignments

Knowing course codes, deadlines

Student, course, assignment, result

chapter 18

modelling rich interaction

18.1 Introduction

Main idea: Traditional HCI models - such as Norman's execution - evaluation cycle - focus on discrete actions like "click a button" or "press a key." But in modern systems, interaction is:

Continuous (gesture movements, real-time video editing).

Multimodal (voice + gesture, touch + haptic feedback).

Context-aware (adapts to location, time, or user state).

Why modelling is important:

Breaks down complex behaviour into understandable components.

Predicts potential problems before development.

Supports communication between different stakeholders (designers, engineers, managers).

Example scenario:

Designing an **AR navigation app**:

User points phone to street.

GPS + camera + orientation sensors combine input.

App overlays arrows in real-time.

Context: Sunlight glare, crowd movement, network speed — all impact interaction.

18.2.1 The nature of status–event analysis

Definition: A way to model a system as a set of **statuses** (conditions) and **events** (changes).

Why this works well:

Fits the way humans think about cause and effect.

Encourages designers to identify every **state** the system can be in.

Makes **feedback gaps** visible.

Key principle: Every event should lead to a **clear and observable status change** for the user.

Design note: Missing or delayed feedback = confusion.

Example:

Status: *Printer ready, Printer busy, Error.*

Event: *Print command, Print completed, Paper jam detected.*

Scenario: Brian checks his watch repeatedly. Alison checks her **calendar**

Abstracted Properties

Status:

Watch → continuous status (time changes smoothly).

Calendar → discrete status (specific dates/events).

Events:

7:35 arrives (time event). Brian leaves (user action event). Watch alarm rings (system event). Alison notices birthday (perceptual event).

Polling:

Brian repeatedly checks his watch. Turning a **status** (time) into an **event** (7:35 reached).

Actual vs. Perceived:

Actual event = 7:35 reached., Perceived event = when Brian notices (7:36).

Granularity:

Watch → minutes scale.

Birthday → yearly scale.

Key Idea for HCI

Status and events interact differently depending on **continuity, perception, and granularity**. Users (and systems) may **poll, perceive, or miss** actual events.

18.2.2 Design Implications

Idea: Applications create **events for users**. The **presentation technique** must **match the timescale** of the event.

Examples

Fast timescale – Critical events:

Coolant failure in nuclear plant → needs **immediate alarm** (seconds).

Email would be too slow → catastrophic.

Slow timescale – Routine events:

Stock of 6mm bolts low → reorder within **days/weeks**.

Red flashing alarm would be **overkill**.

Everyday mismatch:

Brian's cinema reminder (short-term) → alarm at 7:35 works well.

Alison's birthday reminder (long-term) → alarm at noon is disruptive.

A calendar notification fits better.

Key Lessons for HCI

Wrong timing can be **dangerous** (too slow) or **annoying** (too fast).

Designers must **predict event timescales** and choose suitable techniques:

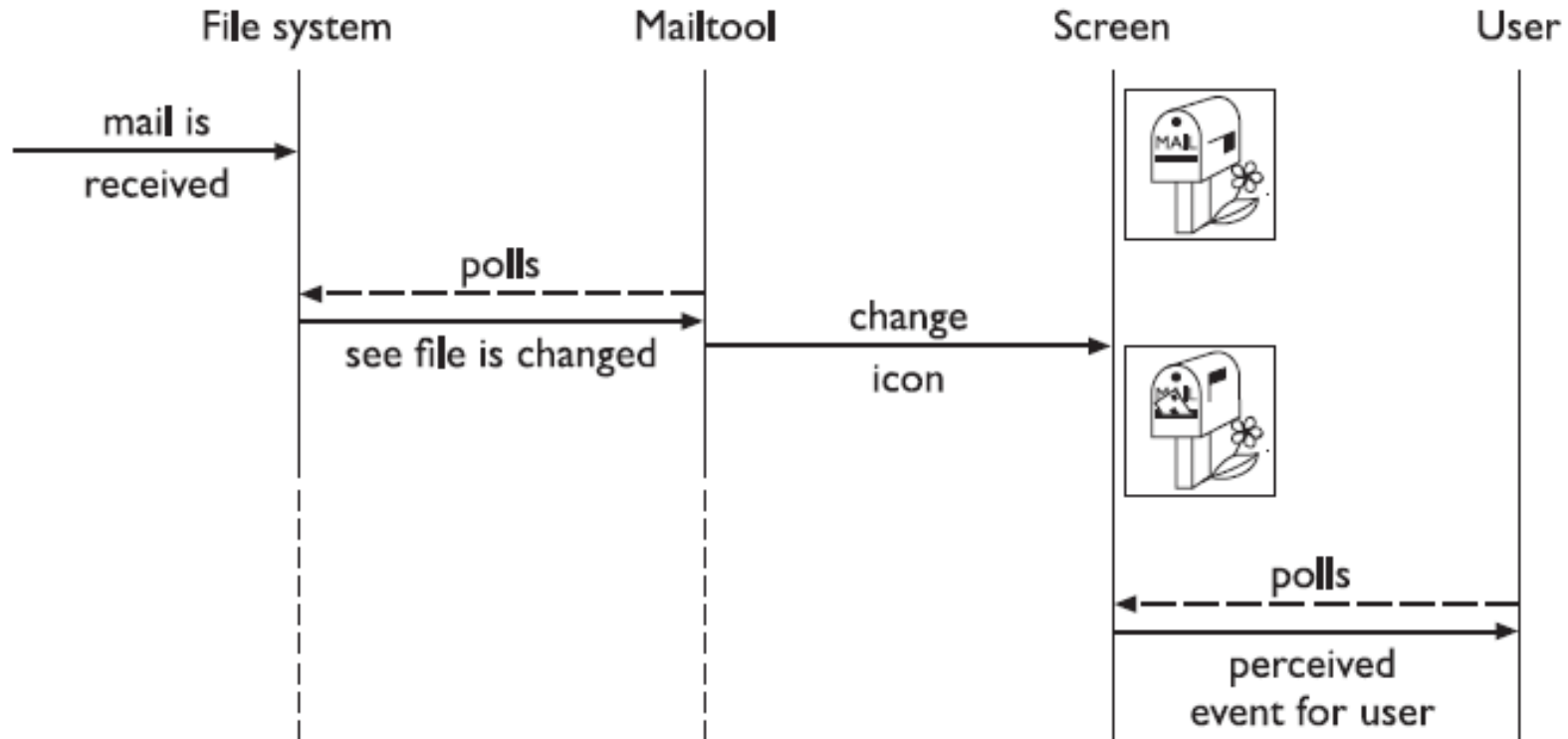
Alarms/alerts → short timescales.

Calendars/schedules → long timescales.

Simply showing info $\neq$ user perceives it at the right time.

18.2.3 Naïve psychology

where the user will be looking, Position of mouse, screen, text insertion point



18.2.5 Example – screen button feedback – Home Work

Problem Statement:

As an HCI analyst, you are tasked to model **rich interaction in MIST UNIPLEX** using **status–event analysis** and apply **design implications** for different timescales of events.

Tasks to do:

Identify statuses and events in UNIPLEX for at least **two scenarios**:

- (a) **Assignment submission workflow**
- (b) **Exam result publication**

Draw a status–event diagram for each scenario.

Example statuses: *Assignment Pending, Assignment Submitted, Deadline Passed.*

Example events: *Student uploads file, Deadline reached, Teacher releases grade.*

Analyze polling vs. event notification:

Do students need to **poll** (check repeatedly) for results/assignments?

Or does the system generate **events/alerts** (e.g., push notifications, emails)?

Discuss timing mismatches (Design implications):

What happens if notifications are **too slow** (e.g., assignment reminder comes after deadline)?

What happens if they are **too fast or disruptive** (e.g., repeated alerts for every minor update)?

Propose improvements in UNIPLEX:

Suggest **better event presentation techniques** for short-term (urgent) events vs long-term (routine) events.

Problem Statement:

Expected Outcomes

Students will demonstrate how **status–event analysis** simplifies complex workflows.

They will understand the importance of **matching notification techniques** to the **timescale of events**.

They will propose design improvements to make MIST UNIPLEX more **user-friendly and efficient**.

(a) Assignment Submission Workflow in UNIPLEX

Statuses

1. Assignment Pending – Teacher has uploaded assignment instructions, awaiting student submission.
2. Assignment Submitted – Student has uploaded file before deadline.
3. Deadline Approaching – Time is nearing submission cut-off.
4. Deadline Passed (No Submission) – Student missed submission.
5. Assignment Graded – Teacher evaluates and uploads marks/feedback.
6. Grade Released – Student can view result.

Events

- Teacher uploads assignment → Status changes from *No Assignment* → *Assignment Pending*.
- System sends reminder (X days before deadline) → Status remains *Assignment Pending*.
- Student uploads assignment → *Assignment Pending* → *Assignment Submitted*.
- Deadline reached → *Assignment Pending* → *Deadline Passed*.
- Teacher grades submission → *Assignment Submitted* → *Assignment Graded*.
- Teacher publishes grade → *Assignment Graded* → *Grade Released*.

Status–Event Diagram (Text Form)

No Assignment

↓ (Teacher uploads assignment)

Assignment Pending

↓ (Student uploads file) -----→ Assignment Submitted

↓ (Deadline reached without submission) → Deadline Passed

Assignment Submitted

↓ (Teacher grades)

Assignment Graded

↓ (Grade released)

Grade Released

Polling vs. Event Notification

- Current practice in UNIPLEX: Students often have to poll (log in repeatedly) to check assignment deadlines, submission confirmation, and grading updates.
- Ideal design: Introduce event notifications (push notification, SMS/email reminders).
 - Before deadline → Reminder alert.
 - After submission → Confirmation alert.
 - After grading → Grade release alert.

Design Implications (Timing Issues)

- Too slow: Reminder arrives after deadline → Student misses submission.
- Too fast/disruptive: Repeated alerts every few minutes → Annoying and distracting.
- Balanced approach:
 - Short-term/urgent → Strong alerts (reminder 24h and 2h before deadline).
 - Long-term/routine → Calendar integration (assignment release dates, grading release).

Proposed Improvements for UNIPLEX

1. Push notifications & emails for deadlines and results.
2. Submission confirmation receipt (so student knows file uploaded successfully).
3. Calendar sync with student device (Google/Outlook Calendar).
4. Progress tracker dashboard showing assignment lifecycle (Pending → Submitted → Graded).

Classroom Problem Statement: Attendance Workflow in UNIPLEX

You are asked to analyze how **attendance** is managed in UNIPLEX using **status–event analysis**.

Tasks:

1. Identify key **statuses**
2. Identify **events**
3. Draw a **status–event diagram** showing the workflow.
4. Discuss whether attendance updates should rely on **polling** or **notifications**.
5. Suggest **improvements** to make the attendance process more efficient and user-friendly.

Classroom Problem Statement: Attendance Workflow in UNIPLEX

You are asked to analyze how **attendance** is managed in UNIPLEX using **status–event analysis**.

Tasks:

1. Identify key **statuses** (e.g., *Class Scheduled*, *Attendance Pending*, *Attendance Marked*, *Attendance Missing*).
2. Identify **events** (e.g., *Teacher opens attendance sheet*, *Student marked present/absent*, *Attendance submitted*).
3. Draw a **status–event diagram** showing the workflow.
4. Discuss whether attendance updates should rely on **polling** (students checking manually) or **notifications** (system alerts).
5. Suggest **improvements** to make the attendance process more efficient and user-friendly.

Expected

Outcome:

Students will understand how to apply status–event analysis to attendance and propose practical design enhancements.